

从 Java 开发者视角读懂 trainng 的 DSH 集成

这套集成的核心，是把 DSH 当作一个长期运行、拥有完整会话能力的 Agent 应用。Fitness 负责训练业务，DSH 负责模型交互和执行过程。两者通过 HTTP、浏览器消息和共享工作区协作。真正值得学习的，是这种职责拆分，以及 Cordis 如何让应用在保留官方能力的同时替换界面、加入业务上下文和自动任务入口。

本文从进程、调用、数据、插件到可靠性逐层展开。前半部分解释现有代码，后半部分讨论怎样扩展、怎样接 Java，以及下一步最值得改进什么。代码片段会明确标注为“现有实现节选”或“建议示例”，避免把设计建议误读成已经上线的功能。

研究基线为 trainng 提交 417e002e08ffdfef4faa38b34b7e66404de0b3f23，实际安装的 DSH 为 0.1.5-rc.2，Cordis 为 @deepseek-ai/cordis@4.0.2；旁边的 deepseek-harness 检出提交为 dd6322d604e00eec1ba5e0c8541159906a21094a。分析以实际安装包为运行接口依据，邻近源码仓库用于理解设计，不把两个快照视为完全相同。核验时间为 2026 年 9 月 14 日。官方 rc.2 发布页也明确标记该版本为预发布版本。1 23

****阅读建议：****第一次按第 1—9 章建立整体认识，再读第 10—14 章的 Cordis 和 Java 扩展；第 15—18 章适合评审可靠性和规划下一阶段。离线 HTML 有目录，PDF 适合平板阅读，所有核心说明与示例均已放在正文，不需要联网追链接才能读懂。

1. 先掌握五个判断

****第一，DSH 不是模型。模型提供生成能力；DSH 负责把模型、工具、会话日志、上下文、审批、浏览器交互和执行循环组织成可运行的应用。接入 DSH，意味着接入一套 Agent 运行时，而不只是换一个 Chat Completion SDK。**

****第二，本项目集成的是完整 Web Host，并定制它的组成。Fitness service 启动 `dsh --profile fitness`；这个 profile 叠加官方 base、官方 web-app 和项目私有插件。应用并没有把 DSH 的 Agent Loop 复制进自己的业务代码。2**

****第三，交互和自动化共用一个 Host，使用不同 Session。用户在 iframe 里聊天；Scheduler 通过受 secret 保护的本地 bridge 入群。两个入口最终使用同一套 DSH Session Controller，但交互 Session、每日定时 Session、手工任务 Session 的业务含义不同。3 4 5**

****第四，模型完成、Agent 空闲、业务提交成功是三个不同事实。自动任务获得 202 后还要等待 Agent 状态；Agent 回到 idle 后，Fitness 还要验证目标文件、档案版本和变更范围。这是整个设计最重要的正确性原则。4 5**

****第五，Cordis 的扩展能力很强，但作用域和生命周期决定扩展是否真正生效。把插件放进 npm 依赖不等于挂载；把配置写在 Host 层不等于改变 preset 内的同名插件；隐藏按钮也不等于撤销执行权限。后文会用实际发现说明这些区别。**

2. 用 Java 概念建立对应关系

DSH / Cordis 概念	Java 开发者熟悉的近似概念	必须保留的差别
DSH Host	一个长期运行的应用进程	同时包含 Agent 运行时和浏览器服务
Cordis Context	ApplicationContext + 服务查询入口	服务查找还受 realm、插件依赖和调用作用域影响
Plugin	配置驱动模块、自动配置单元	挂载和卸载都有真实运行时语义
Service	受容器管理的能力接口 / 实例	常通过 <code>ctx.xxx</code> 按服务名获取
<code>inject</code>	显式依赖声明	依赖消失会影响消费者生命周期，不只是启动时注入
<code>ctx.effect()</code>	资源申请与销毁钩子配对	必须能撤销注册、监听、计时器等副作用
<code>waterfall</code>	FilterChain、Interceptor 的 around 调用	需调用 <code>next()</code> 才继续；并非把上个返回值传给下个函数
profile	应用组装配置	决定 Host 装哪些 bundle、使用何种应用入口
agent preset	某类 Agent 的能力配置	与 Spring profile、Host profile 都不是同一个概念
Session	持久会话聚合及事件流	不等于线程、HTTP Session 或数据库事务
Agent	Session 对应的活跃执行句柄	可以恢复；多个 Agent 可存在于同一个进程
Tool	模型可调用的能力接口	有 schema、执行策略、返回值与展示协议
Skill	按需加载的操作知识包	是指导材料，不是强制授权策略
Session projection	事件流派生的读模型	聊天读模型与 Fitness 训练读模型各有事实来源

这个对应表只用于理解，不能据此把 Node 运行时当作 JVM。Node 的多个异步任务能在同一进程交错执行；“没有多线程共享字段”不代表“没有并发问题”。两个 HTTP 请求都可以在第一次 `await` 后让出执行机会，随后同时进入尚未完成的逻辑。

例如 bridge 的 `Map.get` → `await prompt` → `Map.set` 就存在这种交错窗口。Java 开发者可以把它看成缺少原子占位的“先查后做”，不能因为 JavaScript 单线程便认定天然幂等。

3. 进程关系：谁启动谁，谁活多久

3.1 正常开发和运行拓扑



这张图描述应用层关系，不承诺操作系统恰好只有三个 PID：npm、watcher、shell、工具和浏览器本身都可能额外产生进程。核心关系是 **Fitness service 是 DSH Host 的父进程，DSH Host 长期存在，Session 不各自启动一个 Host**。1 2 3

`DshWebHost.start()` 使用 `spawn(process.execPath, args, options)`，相当于 Java 的 `ProcessBuilder`。执行的是当前 Node 可执行文件和项目安装的 DSH CLI 文件；它不是依赖用户全局 PATH 上某个不确定版本的 `dsh`。`cwd` 指向个人工作区，`DSH_HOME` 指向该工作区的 `runtime/dsh`。

DSH CLI 随后在自己的进程里装配并启动 `profile`，没有在这条普通 Web 启动路径上再创建一个“每会话子服务”。执行 Bash、CLI、语言服务器等能力时仍可能生成其他进程；内置 in-process 子 Agent provider 也说明，名字叫“子 Agent”不一定代表操作系统子进程。12 19

3.2 三个不同生命周期

对象	创建时机	结束或失效时机	重建是否意味着业务数据丢失
Fitness service	应用启动	应用停止或崩溃	正式文件仍在工作区
DSH Host	service 启动，或部分入口触发重试	service 关闭、Host 故障	日志与状态可能支持恢复，内存 Map 会丢失

对象	创建时机	结束或失效时机	重建是否意味着业务数据丢失
Session / Agent	bootstrap、用户新建、自动任务 admission	会话可持久存在，活跃执行可空闲或释放	与 Host 内存、训练文件分开考虑
iframe 页面	首次打开 Agent 界面	重试重载、页面刷新、主页面关闭	官方 Session 历史与页面实例分离

关闭聊天面板只是隐藏 iframe，不卸载已创建的页面。因此输入草稿和连接可以保留。浏览器彻底刷新之后，页面重新连接并恢复官方会话列表；这不是 Fitness 自己重新组装消息列表。7

3.3 启动就绪不等于每种功能都健康

Host wrapper 从 stdout 中提取 `dsh web: http://...`，据此把状态从 `starting` 改成 `ready`。这证明 Host 宣布了访问入口，不证明模型凭据可用、某次任务可成功，或浏览器内所有功能已经准备好。

至少有四种就绪需要分别理解：

- 1. service 在监听；
- 2. DSH 打印访问 URL；
- 3. 浏览器 Connection 与 Session 列表进入 ready；
- 4. 模型请求和业务写入可以完成。

当前 Docker 健康检查只访问 Fitness `/api/health`。它不是四种就绪的完整检查。2 7 11

4. 一次启动到底装配了什么

4.1 项目安装依赖与运行时 profile 是两件事

`package.json` 的关键部分是：

```
{
  "@deepseek-ai/dsh": "0.1.5-rc.2",
  "@ai-fitness-os/dsh-fitness-automation-bridge": "file:dsh-fitness/automation-bridge",
  "@ai-fitness-os/dsh-fitness-surface": "file:dsh-fitness/surface"
}
```

这是“代码能被解析”的前提。真正运行哪些插件，仍由 `profile` 配置决定。类似 Java 应用中 `jar` 在 `classpath` 上，只是具备加载条件；有没有注册对应 `Bean`，是另一个问题。1

Host 每次启动都会在 `$DSH_HOME/profiles/fitness` 生成配置，核心结构如下：

```
WORKSPACE_ROOT/
runtime/dsh/profiles/fitness/
  package.json
  cordis.patch.yml
  node_modules/@ai-fitness-os/
    dsh-fitness-automation-bridge → 应用仓库内插件目录
    dsh-fitness-surface           → 应用仓库内插件目录
```

生成的 `profile` 声明两个 `bundle`，并指定启动时加载 `patch`：

```
{
  "dsh": {
    "profile": {
      "bundles": ["@deepseek-ai/dsh-base", "@deepseek-ai/dsh-web-app"],
      "patchReload": "startup"
    }
  }
}
```

因此，直接编辑生成目录的 `cordis.patch.yml` 不适合作为长期定制方式：下一次启动会被项目源配置重新生成。应该修改 `dsh-fitness/profile/cordis.patch.yml`，或在明确管理的更高层配置中做覆盖。2

4.2 配置层叠是“组合插件树”

默认组合顺序为：

空插件列表

- profile 声明的 bundle，按声明顺序应用
- profile 自己的 cordis.patch.yml
- DSH_HOME/cordis.patch.yml
- 命令行 --patch 的 overlay

后面的层可以针对已有 `id` 修改配置，或者插入新插件。重要细节是：**对一行的 config 做 patch，是替换该行整个 config，不是任意深度递归合并。**因此覆盖 webserver 时，项目同时明确列出了 host、port 和压缩参数。12

```
# 现有配置节选
- id: webserver
  config:
    host: "__FITNESS_DSH_BIND_HOST__"
    port: !!js ctx.webStartup.port ?? 3080
    compression: gzip
    compressionLevel: 1
    compressionThresholdBytes: 1024

- insert:
  - id: fitness-automation-bridge
    name: "@ai-fitness-os/dsh-fitness-automation-bridge"
  - id: ui-fitness-surface
    name: "@ai-fitness-os/dsh-fitness-surface"
```

`id` 是配置树里的行标识；`name` 是需要解析的插件模块名；`config` 是传给插件的配置。三者不是同一个命名空间。也不要把 `id: webserver` 与注入服务名 `webServer` 混为一谈。

`!!js` 是 DSH 使用的 Include / Loader 表达式能力。它在对应配置环境里求值，不是普通 YAML 的静态字符串。可执行配置应视作受信代码，而不是开放给普通最终用户的任意表达式字段。12 13

4.3 为什么 package 链接对 Docker 很重要

两个私有包通过 `file:` 依赖安装，又由 `profile` 目录链接到应用源码。Docker 最终镜像必须同时保留这些插件目录和依赖解析所需结构。只复制 `dist` 页面或 TypeScript 编译输出，会让 Loader 找不到插件。

现有 Dockerfile 在安装依赖前复制插件 `package.json`，运行镜像再复制整个 `dsh-fitness` 目录。这是集成方式的一部分，不是普通静态资源打包的细节。11

5. 最容易漏掉的一层：Host profile 与 Agent preset

5.1 Host 决定应用，preset 决定 Agent 能力

在 rc.2 Web 组合中，Host 保留共享注册表、持久化、模型路由、sandbox / approval、控制器等服务。许多面向模型的工具、指令发现和其他能力则通过 Agent preset 装配。

```
Host profile: fitness
├─ 共享的 Session / Tools registry / LLM / Credentials / Controllers
├─ Fitness bridge、全局教练 prompt、Web Surface
└─ Agent preset registry, 默认 standard
    └─ standard 的 standing scope
        ├── persona / agent-instructions
        ├── read、write、bash、skills 等工具贡献
        └─ 其他计划、工作流和压缩相关贡献
    └─ 使用 standard 的各个 Agent 通过 scope parentage 加入
```

实际安装包的 preset 实现会为一个 preset 建立共享的 standing composition，再让 Agent 通过作用域关系加入。不要把它想成“每新建一条 Session，就重新启动整个容器并创建一份全部插件单例”。与业务相关的状态仍必须按 Agent / Session 区分。17

同样，不能因为两个 Agent 采用同一个 preset，就认为它们应该共享一份可变训练状态。共享的是能力注册与组合，训练归属仍要来自当前 Agent、workspace 和业务操作。

5.2 本项目存在一个具体的配置生效偏差

项目 Host patch 写了：

```
- id: agent-instructions
  config:
    instructionFileCandidates: [AGENTS.md]
```

但官方 dsh-web-app 先把 Host 层这行设为 disabled: true。Fitness patch 只替换 config，没有重新启用它；实际执行的指令插件来自 standard preset，它只配置 maxBytes: 65536。安装包中该插件默认的候选文件是 AGENTS.md 和 CLAUDE.md。2 17

隔离配置探针得到的 Host 组合结果是：

```
- id: agent-instructions
  name: '@deepseek-ai/dsh-agent-instructions'
  config:
    instructionFileCandidates:
      - AGENTS.md
  disabled: true

- id: agent-presets
  name: '@deepseek-ai/dsh-agent-presets'
  config:
    default: standard
```

所以，“项目意图只发现 AGENTS.md”是明确的设计要求；“当前这个 Host patch 已经保证只发现 AGENTS.md”则不成立。这不是 AGENTS.md 不能读取，而是排除其他候选文件的控制没有作用到实际 preset 那一行。已有真实 Host 测试验证了读取 AGENTS.md，没有验证 CLAUDE.md 不会进入上下文。21

建议建立部署拥有的 `fitness-coach` preset，在其中明确设置候选文件和工具范围，再以默认选择或 create 时的 `agentPreset` 绑定使用它。不要简单把 Host 同名插件重新打开，否则可能同时存在全局贡献和 preset 内贡献，还要继续证明覆盖、去重和作用域行为。

5.3 一个有价值的产品扩展方向

未来可以让交互教练和每日自动任务使用不同 preset：交互教练需要解释、提问和建档；自动任务只需要读取事实、生成目标日期草稿和提交校验。Host 和会话基础设施仍可共用。

这有助于减少无关工具、缩小权限、让自动任务更稳定。但当前 bridge 没有把 `agentPreset` 作为入参传给 create，交互 bootstrap 也没有显式指定它；这个方案需要实际改造，不能只新建一个 preset 文件就认为任务已经切换。

6. 用户发一句话，会穿过哪些调用层

6.1 打开页面的链路

```
Fitness AgentChat.expand()
  → GET Fitness /api/model/settings
  → GET Fitness /api/dsh-web
  ← { status: "ready", url: DSH 访问入口 }
  → iframe 加载 DSH 根页面
  → DSH 官方认证、Client Modules、Connection 和 Session 机制
  ↔ parent/iframe 的 fitness.surface.connect / ready 握手
  → 展示官方 conversation
```

Fitness 首先检查凭据是否已配置；这不是向模型发一次连通性测试。Host URL 中的启动 token 交给 DSH 官方浏览器认证机制处理。iframe 内再使用官方 Session Controller 对应的浏览器能力连接会话。3 7

rc.2 的浏览器 unary RPC 走 HTTP POST，流式 Remote 通过 Gateway 的 `/api/remote.mux` WebSocket 承载；这与 Fitness 自己的 `/api/data-events` SSE 是不同通道。某些配置注释仍使用早期 Fetch/SSE 表述，应以实际 Connection 和 Gateway 实现为准。18

6.2 Surface 替换了什么，保留了什么

项目关闭默认 layout、sidebar、官方品牌，以及几个设置入口，再由私有 Surface 提供 root、main、rightbar 和 overlay 插槽。它仍让官方 conversation 渲染消息、工具和其他会话内容。2 8

Surface 包的 Host 入口非常简单：

```
// 现有 Host 入口
export function apply() {}
```

它的重要作用是包里的 `dsh.client.platform: "web"` 元信息和浏览器模块。浏览器部分通过 `window.__ModuleLoader__.load(...)` 加载，使用 DSH 提供的 React 和 Client 服务，注册自己的布局。这解释了为什么“Host 文件几乎为空”仍然能改变整个界面。

这种做法类似替换应用的页面布局模块，而不是重新实现消息协议。父页面和 iframe 各有 React 运行环境、状态和 DOM。父页面不能直接把 iframe 里的 `ctx.sessions` 当自己的对象调用。

6.3 最小消息桥而不是消息镜像

父页面使用 `postMessage` 做布局协调：

- `fitness.surface.connect / ready / collapse`：连接和收起；
- `fitness.agent.run-state`：当前会话是否运行中；
- `fitness.history.state / close / refresh / select`：历史导航。

父页面校验来源 `origin` 和 `iframe window`；历史数据另有 `schema` 校验。发送方指定目标 `origin`。历史面板只投影 ID、标题、更新时间和运行状态，没有把完整聊天内容保存到 Fitness。7 8

有一个细节要注意：子页面通过实际 `window.parent` 的 `connect` 消息学习 `parent origin`，并非启动时就持有固定的生产域名白名单。它限制了消息来源窗口和 HTTP(S) `origin`，但生产环境仍需配合允许嵌入来源、认证与部署策略，不能仅凭“没有使用星号”断言全部边界已经封闭。

6.4 消息进入 DSH 后的运行过程

官方 Session prompt

- 创建 / 恢复活跃 Agent
- inbox 入队
- 一次 turn 开始
 - 读取本次上下文，组装 prompt 和工具 schema
 - 模型请求
 - assistant 文本 / 工具调用
 - 执行工具，记录结果
 - 若还需继续，进入下一个 step
- turn 结束，Agent 最终可能回到 idle
- 官方会话投影和浏览器展示更新

一个 step 大体是一轮模型请求及其工具执行；一个 turn 可以包含多个 step。调用一次 prompt，不能假定只产生一次模型请求。模型请求、工具反馈、上下文压缩和其他运行机制都可能增加实际调用次数。12 16

Session 的持久日志承担会话回放；实时 `agent/status` 表示活跃执行情况。Java 开发者可以类比“业务事件记录”和“内存中的任务状态通知”，但不能把二者互相替代。

7. 每日自动计划：一条完整的执行链

7.1 Scheduler 先决定是否需要模型

Scheduler 属于 Fitness，默认每分钟 tick。它根据启用开关、IANA 时区、本地时间和 occurrence 判断是否执行。调度状态有 cursor、claim、retry 和 last_run；实际实现用进程内 Promise 队列串行化该 Scheduler 的工作。5

调用模型前，preflight 依次判断：

- 1. 已有当日 workout：already_completed；
- 2. 已有当日计划且校验通过：already_exists；
- 3. 没有已确认档案：profile_required；
- 4. 没有覆盖目标日期的已确认 program：no_active_program；
- 5. 周计划该日为恢复：recovery_day；
- 6. 否则才调用 Agent。

因此 status: succeeded 不只表示“生成了新计划”，也可能是上述正常无需执行的领域结果。客户端应该读 outcome.code，不能只按 success 显示“今天计划已生成”。

7.2 三类标识，各有职责

标识	现有生成方式	职责
occurrence	日期T本地时间[时区]	表示一次业务调度发生
scheduled Session ID	fitness-daily-YYYY-MM-DD	某日定时任务的会话归属
manual Session ID	fitness-daily-日期-manual-UUID	隔离一次手工触发
Scheduler run_id	Scheduler 内生成 UUID	一次业务执行及运行日志
bridge runId	AgentRuntime 内另生成 UUID	bridge 状态查询和 Map key
requestId	fitness- + bridge runId	DSH prompt 的请求去重身份

这里有一个容易漏掉的事实：Scheduler run_id 与 bridge runId 是两次独立生成，当前接口没有把它们直接串成一个统一 trace。排障时不能拿 Scheduler 日志里的 run_id，理所当然地当成 bridge status 的 runId。4 5

7.3 bridge 的 HTTP 调用

下面是**现有协议的手工演示**，会真正入队一条 Agent 消息；阅读报告不需要执行它。`REPORT_BRIDGE_SECRET` 必须是当前 Host 的独立 bridge secret，不能拿浏览器 URL token 或 DeepSeek Key 代替。

```
curl -i http://127.0.0.1:3080/fitness-automation-bridge \
-H 'Content-Type: application/json' \
-H "x-fitness-bridge-secret: ${REPORT_BRIDGE_SECRET}" \
--data '{
  "sessionId": "fitness-daily-2026-09-14",
  "runId": "example-run-001",
  "requestId": "fitness-example-run-001",
  "message": "检查当前档案和目标日期计划，说明还缺少哪些输入。"
}'
```

正常 admission 响应类似：

```
{"ok":true,"accepted":true,"sessionId":"fitness-daily-2026-09-14"}
```

bridge 用恒定时间比较检查 secret，限制请求体为 64 KiB，验证四个字段为非空字符串，随后执行：

```
// 现有调用节选
await ctx.sessionController.create({
  sessionId: input.sessionId,
  cwd: process.cwd()
});

await ctx.sessionController.prompt({
  sessionId: input.sessionId,
  requestId: input.requestId,
  mode: "queue",
  content: [{ type: "text", text: input.message }]
}, admission.signal);
```

控制器的 create 可以幂等创建或接纳既有普通 Session；prompt 把消息送入官方 Agent inbox。`queue` 在当前控制器中走 followup；另有 steer 路径，语义不同。桥接代码没有自己实现另一个 Agent Loop。6 16

7.4 202 之后如何判定完成

`DshHostAgentRuntime.run()` 每 250 ms 查询：

```
GET /fitness-automation-bridge/status?sessionId=...&runId=...
```

bridge 内存记录从 `queued` 到 `running`，再在观察到运行后进入 `idle`。`runtime` 看见 `idle` 就返回空的 `finalResponse` 和 `notifications`；`Scheduler` 不读取模型文本来判断成功。4 6

`Scheduler` 接着核对运行前的档案 `revision`，校验目标计划，并计算前后文件变化；只有这些检查通过才记 `created`。因此最后一句“已经完成计划”对业务成功没有决定权。真实的权威是目标文件和确定性程序。

7.5 幂等有多层，不能合并成一个“支持去重”标签

bridge 的 `accepted` Map 按 `(sessionId, runId)` 缓存成功 admission；它是进程内状态，Host 重启会丢失。

DSH 控制器还会根据 `requestId` 检查待处理 inbox 和持久 `user/message` 事件，已发现相同请求就直接返回 `accepted`。这比只看 bridge Map 强，但不意味着端到端 `exactly-once`：请求还涉及异步 admission、Host 恢复、业务文件提交和调度日志多个边界。16

`Scheduler` 的每次 `runtime.run()` 又会创建新的 bridge `runId` / `requestId`。所以 `Scheduler` 重试并不自动复用之前的 DSH 请求身份；业务上的防重复仍主要来自 `preflight` 和正式提交逻辑。

一个合理的下一步，是让 `Scheduler` 持久化并传递 `occurrence`、业务 `runId`、admission `requestId`，明确“同一次网络重传”和“新的业务重试”何时复用标识，而不是在两层各自生成 UUID。

8. 文件、校验和事务：谁拥有训练事实

8.1 工作区中的状态分层

应用安装目录	个人 WORKSPACE_ROOT
resources/fitness/	AGENTS.md
templates/	fitness/
examples/	profile.yaml
docs/	programs/
dsh-fitness/	plans/
	workouts/
	metrics/
	config/
	settings.yaml
	dsh-credentials.yaml
	runtime/
	onboarding/ 草稿、受控提交锁
	automation/ 调度状态和运行日志
	dsh/ Host / 会话相关状态

默认 workspace 为应用目录下 `.workspaces/default`；`WORKSPACE_ROOT` 可以显式切换。代码强制 `fitness`、`config`、`runtime` 和 `DSH_HOME` 的位置关系，避免同时设置互相冲突的旧环境变量。9

训练 YAML 与 DSH 会话日志分别保存不同事实。聊天中讨论过某个重量，不等于 `workout` 中已经记录了真实完成重量；模型想出了明天的计划，也不等于用户今天完成了训练。10

8.2 CLI 是领域服务边界

Agent 读取业务契约
→ 在 <code>runtime/onboarding</code> 写草稿
→ 调用 Fitness CLI
→ <code>schema</code> 、路径、档案 <code>revision</code> 和确定性计算
→ 正式文件发布

对 Java 开发者来说，CLI 扮演一个可跨进程调用的 Application Service。它可以被 shell 工具调用，但领域规则集中在 `data-store.ts`、`onboarding.ts` 和 `shared schema`；没有散落在自然语言里让模型自由解释。

例如 `finalizePlanFile()` 检查目标必须在 `plans` 内，日期和文件名一致，不能带实际 `workout` 专用字段，也不能由草稿预填 `computed` 结果。程序计算刺激数据，再进入 `withProfileRevision()` 保护的提交阶段。10

```
# 现有开发入口示例；仅在打算提交所指草稿时执行
WORKSPACE_ROOT=/path/to/workspace npm run fitness -- \
  finalize plan /path/to/workspace/runtime/onboarding/plan-draft.yaml

WORKSPACE_ROOT=/path/to/workspace npm run fitness -- validate all
```

这些命令应从应用根目录运行；部署环境的工作区 AGENTS.md 会写出相应实际 CLI 路径。不要在 Docker 生产环境直接假定存在开发依赖 tsx。

8.3 这里有锁，但不是任意操作的完整事务

现有代码包含真实的跨进程锁：例如 profile 更新与受支持的计划 / program 发布共用 `profile-update.lock`，申请时用 `flag: "wx"`；配置保存也使用锁目录。计划修订还可以核对目标计划 SHA256。¹⁰

因此，说“整个系统完全没有锁”是不准确的。但这些锁只保护经过指定入口的操作。Agent 若直接通过通用 shell 写文件，不会自动进入同一事务；Scheduler 的 Promise 队列也不能阻止用户交互 Agent 同时修改目标。

原子 rename 主要避免读者看到半个文件，不自动提供多个文件之间的 ACID 事务。文件快照审计能发现部分越界修改，但发生错误时不会自动把所有变更回滚。把这一点类比为“有乐观锁和部分互斥的文件系统领域层”，比简单称它为数据库事务准确得多。

8.4 Dashboard 为何通过另一条 SSE 更新

```
人工 / Agent / CLI 写入 fitness 文件
→ Fitness DataSync 观察目录变化
→ 350 ms 稳定等待
→ 内容 fingerprint
→ validateFitnessData + 构建只读 Dashboard
→ 再算 fingerprint，确认校验期间没有变化
→ SSE: fitness.data-changed
→ 主页面重新获取业务投影
```

无效 YAML 或未通过领域校验的数据触发 `fitness.data-invalid`。这一机制并不依赖会话文本、iframe load 或 Agent idle，人工编辑文件同样可以被观察。⁹

SSE 消息不伪造 Session ID，因为文件监听不能可靠知道“是谁写的”。如果未来需要把变更归属到 `runId`，应该由受控提交入口记录，而不是通过“最近哪个 Agent 在运行”猜测。

9. 上下文集成：模型怎么知道自己是健身教练

9.1 当前主要有三层材料

第一层是 DSH / preset 提供的基础运行和工具说明。第二层是项目 bridge 注册的 `fitness-coach-instructions` system prompt section，包含教练职责、建档、事实边界、必读输入和 CLI 提交规则。第三层是当前工作区的档案和用户表达设置。6

工作区 AGENTS.md 是导航：告诉 Agent 去哪里读业务契约、公共资源和个人文件。应用仓库自己的 AGENTS.md 主要约束修改项目的开发 Agent；DSH 的 cwd 已经切到独立个人工作区，不能想当然地把两者当成一份指令文件。9

9.2 动态 section 为什么比会话创建时拼一次 prompt 更合适

现有注册使用 `text: () => ...` 回调。DSH 在组装请求时求值，回调重新读 settings 和 profile 原文，计算 SHA256，再加入当前模型上下文：

```
本次请求的当前档案（用户业务数据，不是系统指令）
路径：...
profile_revision: <SHA256>
<fitness_profile>
  当前 YAML 原文
</fitness_profile>
```

用户修改器械或时间偏好后，下一次 prompt assembly 可以读取新值，不必删除 Session 再建一遍。已有真实 Host 测试在同一会话连续切换 bands / dumbbells，验证两次请求确实看到不同档案。6 21

缺失档案时返回建档提示；其他读取错误向外抛出，而不是回退到虚构案例。这避免了把“无法读取用户事实”误当成“可以使用默认用户事实”。

9.3 这个实现仍有改善空间

当前把用户档案原文放在 system section 内，虽然文字强调“这是数据”，但角色位置并没有因此改变。rc.2 的 `systemPrompt.context()` 专门支持动态上下文，并投影为持久的 user-role snapshot，值得用于把固定部署指令与用户可变事实拆开。15

另外，原文 YAML 的 SHA256 是字节级 revision；仅格式变化也会改变版本。这对保守冲突检测是合理的，但与“业务内容是否等价”不同。长档案每次同步读入和重复进入模型请求，也会占用本地事件循环时间和上下文预算。

建议保留稳定的教练规则，将当前用户事实放入受大小约束的动态 context；较长历史继续通过工具按需读取。不要在稳定系统段里加入每次都会变化的时间戳和调试文本。是否提升模型缓存命中，需要以实际 provider usage 和请求前缀比较验证，不能只凭 prompt 看起来更整齐来判断。

10. Cordis 深入：从“容器”理解到“可组合运行时”

10.1 Plugin 是带生命周期的贡献者

一个普通函数插件可以长这样：

```
// 建议示例：独立部署规则插件，可作为外部 ESM 包的入口。
export const name = 'fitness-deployment-rules';
export const inject = ['systemPrompt'];

export function apply(ctx) {
  ctx.effect(() => ctx.systemPrompt.section({
    name: 'fitness:deployment-rules',
    order: ctx.systemPrompt.getSectionOrder('DEPLOYMENT_PERSONA_SUFFIX') + 2,
    text: '训练计划是建议；只有用户确认的实际结果才可记录为 workout。'
  })), 'fitness deployment rules');
}
```

这段插件没有拥有一个独立进程，也不需要修改 Agent Loop。它要求 systemPrompt 服务，在激活时贡献一个段落，卸载时撤销该段落。section() 自身已经返回 Cordis effect disposer；显式 effect 在这里展示资源归属，也与项目现有写法一致。13 15

函数插件应保留命名导出的 apply、inject、Config 等协议字段；Service 类插件通常默认导出类。不要不加区分地为函数插件加 default export，导致 Loader 选择默认导出后丢失命名空间上的依赖声明。DSH 仓库对此有专门约定。13

10.2 Service 是能力，而非任意工具函数集合

Java 中可以定义 FitnessDataService 接口，再由本地文件或 HTTP 实现它。Cordis 也可以这么组织：服务声明提供稳定的 ctx.fitnessData，不同 provider 实现能力，模型 Tool 或 Host route 作为 consumer 调用服务。

```
服务定义 FitnessData
├── 本地 provider: 复用现有 data-store / CLI
├── 未来 HTTP provider: 调用 Java 领域服务
└── consumers
    ├── fitness_validate_plan 工具
    ├── 某个受认证业务 route
    └── 管理 / 诊断插件
```

三者分离的收益在于：模型不需要知道训练数据从 YAML 迁到了 Java 服务；UI 也不需要理解底层模型 provider。DSH 的 LLM、shell、文件系统和 subagent 能力都体现了类似模式。12

10.3 inject 不只是自动装配时查一次

如果一个插件声明 `inject: ['webServer', 'sessionController']`，Cordis 在这些依赖可用之前不执行它。依赖被卸载时，消费者的生命周期也受影响；重新可用后可重新激活。配置行的排列顺序不是解决依赖问题的主要手段。13

与 Spring 相比，它更强调“服务和贡献可以在运行中消失”。消费者不能缓存一个永远有效的全局服务引用，又在 provider 被替换后继续使用。

必需服务应显式注入；可选服务在使用点 `ctx.get('name')` 查询。项目 bridge 声明了五个依赖，却直接使用 `ctx.agents.get(...)` 而没有将 agents 列入 inject。现有组合因为 controller 自身依赖 agents 而能工作，但这是对当前依赖拓扑的隐含依赖；建议将直接必需服务明确列出。6

10.4 effect 的本质是资源所有权

以下动作都产生副作用：注册 HTTP route、添加 prompt section、注册 tool、监听事件、监听文件、启动计时器、注入 UI slot。

```
// 建议示例：一般资源生命周期模式。
ctx.effect(() => {
  const resource = acquireResource();
  return () => resource.close();
}, 'named resource');
```

`acquireResource` 在这里是说明用的占位函数。关键是一次 acquire 配一次 release，且归属明确。`ctx.on()` 等框架 API 已自动管理监听清理；手动创建的 Node 计时器、外部连接和 pending operation 仍需要自己纳入生命周期。

如果只注册不清理，开发热重载会出现重复 route、重复 prompt、重复事件监听；它们常常不是首次启动时暴露，而是在第三次改代码后才暴露。这相当于 Java 中 Bean 销毁时忘记关闭线程池或数据库连接，但这里还包括注册表贡献本身。13

10.5 HMR、配置重载、动态读文件不是同一回事

本项目 profile 设为 `patchReload: startup`，所以不能假定保存 Host patch 就会实时改插件树。教练 settings 和 profile 又会在请求组装时读取，所以这些数据不必重启 Host。Skill 文件发现还有自己的 watcher。2 19

这三种变化路径应分别说明：

变更对象	当前或框架行为	实务判断
Fitness Host patch	启动时应用	重启 Host 并验证生效树
固定插件代码	不能从 startup profile 推断自动 HMR	按构建 / 重启流程验证
settings.agent.instructions	每次 assembly 读取	下一次请求组装可见

变更对象	当前或框架行为	实务判断
profile.yaml	每次 assembly 读取、带 revision	下一次请求可见，提交仍需版本校验
skill 目录和正文	provider 有独立发现 / 读取机制	是否可见还取决于实际 preset 的发现根
Agent preset	有自己独立的挂载与刷新机制	不与 Host patchReload 混为一谈

10.6 realm 与 scope：两类容易混淆的隔离

Service realm 解决“同一个服务名能否在不同插件组里有不同实例”。比如两个能力组都需要某个可变服务，可以用 `isolate` 给它们独立解析域。隔离的是服务查找，不是操作系统权限。

Agent scope 解决“同一个注册表中的哪些贡献对当前 Agent 可见”。比如所有 Agent 共享 tools registry，但某个 preset 贡献的工具只在对应作用域链上可见。作用域还影响事件过滤和 prompt 段落覆盖。14 17

进程隔离	OS / 容器 / 用户 / sandbox
服务解析隔离	Cordis realm / isolate
能力可见性	Host → preset standing scope → Agent scope
业务数据隔离	workspace、数据权限、提交规则

这四层不能互相代替。给插件配置了 `isolate`，不代表它不能访问本机文件系统；每个人一个 Session，也不代表实现了多租户授权。一个高权限原生插件仍可能通过 Node API 做任意其进程有权做的事。

11. 事件扩展：什么时候观察，什么时候拦截

11.1 五种派发语义

模式	行为	类比与常见误用
emit	同步调用监听者，不等待其 Promise	适合轻量观察；异步监听不自动获得完成保障
parallel	并发等待监听者完成	扇出任务；不是多个监听者共享一个可变对象的安全保证
serial	依次 await，遇到 bail 值就停止	不是一定跑完全部 handler
bail	同步执行，遇到 bail 值就停止	同步策略链
waterfall	around 链，由 next() 继续	类似 FilterChain；忘记 next 会截断后续

当前 Cordis 判断 bail 的条件是返回值既不是 `null`，也不是 `false` 或 `undefined`。因此返回 `0` 或空字符串也会停止 serial / bail 链。写插件时最好返回协议要求的明确值，避免无意把监控函数返回值变成决策。14

waterfall 本身只是返回最外层回调的结果；具体 DSH 事件可以约定返回 Promise。把它概括成“完全同步事件，不能 async”也不准确。真正应遵循的是该事件的类型声明和调用点。

11.2 工具执行管线

模型提出 tool call	
→ 输入解析与参数校验	
→ tools/pre-execute	可扩展允许 / 拒绝 / 询问策略
→ tools.guard	最终单调拒绝，不提供 allow 翻转
→ tools/execute	around 执行包装
→ 工具实际 body	
→ tools/post-execute	处理返回结果与附加上下文
→ 结果规范化与冻结	
→ tools/result	最终结果观察

具体内部步骤包括取消处理、调度和输出校验，图中保留扩展者最关心的环节。`guard()` 只能增加拒绝，后续监听者不能再把拒绝改为允许，因此适合明确的不可绕过执行限制。15

`tools/post-execute` 适合校验返回结果或控制模型能看见的内容，但执行后的拒绝不会自动撤回已经写入的文件或已经发出的网络请求。机密数据也不能只改人类可读 content，却保留原 canonical value 给程序化调用方读取。

11.3 一个正确的 around 观察例子

```
// 建议示例：不改结果的耗时观察；生产中替换为受控遥测出口。
export const inject = ['tools'];

export function apply(ctx) {
  ctx.on('tools/execute', async (exec, next) => {
    const started = performance.now();
    try {
      return await next();
    } finally {
      console.info(JSON.stringify({
        event: 'fitness.tool.elapsed',
        tool: exec.name,
        sessionId: exec.agent?.id,
        elapsedMs: Math.round(performance.now() - started)
      }));
    }
  });
}
```

它只记录标识和耗时，没有默认输出完整参数、档案或模型文本。 `return await next()` 很重要：如果直接不等待就进入 `finally`，记录的可能只是 Promise 创建时间。这个示例度量 `around-dispatch` 阶段，不包含完整 `admission`、`pre-policy` 和 `post-policy` 耗时，不能命名为“端到端任务耗时”。

11.4 Session 事件与实时通知怎么选

当事实需要刷新后恢复、会话回放或参与模型历史时，优先使用正式 Session 事件和 projection 机制。只观察正在发生的运行过程，可使用 `agent / capability` 实时事件。不要把所有临时 UI 状态都永久写入会话，也不要把需要恢复的事实只保存在 Map。12

新增持久事件还意味着版本协议责任：旧版本是否识别、是否允许忽略、projection 如何重放，都需要明确处理。TypeScript 的 `declaration merging` 只是编译期可扩展接口，不能自动让另一个旧进程理解新事件格式。

12. DSH 扩展点地图：充分利用，而非全部启用

要解决的问题	优先扩展点	对 <code>trainng</code> 的价值与限制
固定教练职责	<code>systemPrompt.section</code>	已使用；保持短而稳定
当前档案、任务事实	<code>systemPrompt.context</code> / 正式上下文事件	可把动态事实从系统指令中拆出
长流程操作知识	Skill provider + <code>SKILL.md</code>	建档、计划修订、训练复盘可按需加载
不同 Agent 的能力集合	Agent preset	区分交互教练和自动计划
确定性业务能力	tools registry + 领域 service	先复用 CLI，明确需求后增加少量结构化工具
访问外部 Java 业务服务	service provider + HTTP 工具适配	不必重写 Agent Loop
模型替换	LlmAdapter 注册	测试已使用本地 fixture adapter
工具执行限制	pre-execute、guard、sandbox provider	prompt 限制不能替代执行限制
请求预算或拦截	agent/request、LLM / 工具包装策略	注意 provider 重试和任务级总预算不同
异步任务	jobs、workflow、subagent	适合推理分工；领域调度仍由 Fitness 拥有
确定性的人类命令	commands	查询状态等操作可避免额外模型请求
外部事件触发会话	webhook 能力及 provider	适合以后接外部系统；目前 bridge 更直接
历史和审计读模型	Session events / projections	保持会话事实在 DSH，不复制消息库
UI 定制	Client slots / theme / locale	当前 Surface 已验证核心方案
专用工具卡片	Web Client <code>tool.call.toolview</code>	需要 Client 注册，不能只实现 Host presenter
跨进程标准工具协议	MCP client	适合跨应用复用和多后端时再评估

这张表是扩展机制选择，不表示每项已经启用或验收。部分机制由特定 bundle / preset 提供，安装了包仍需确认实际挂载、依赖和可见性。12 15 19

12.1 Skill 很适合业务知识，但发现路径要显式设计

DSH 文件系统 Skill provider 支持目录包 `<name>/SKILL.md` 和根目录下的独立 Markdown 文件。默认可能扫描项目 `.dsh/skills`、`.agents/skills`、自定义路径以及用户级路径。项目根按 Git 祖先或 `cwd` 推导。19

Fitness 把 `cwd` 切到个人 workspace，因此应用仓库中新加的 skill 不一定自然被发现。建议将公共训练 skills 放到应用维护的只读目录，并在实际 fitness preset 的 provider 配置中指定 `customSkillDirs`。只有不希望继承默认来源时，再明确设置 `includeDefaultRoots: false`。

适合拆出的 skill 可以是“建档”“根据当前 program 生成每日计划”“修订既有计划”“根据用户报告记录实际 workout”。它们应引用唯一的领域契约和 CLI，不复制一份可能漂移的 schema 或计算公式。

Skill 负责告诉模型怎么做；CLI 和工具 guard 负责哪些操作能被接受。自动任务的“只能写目标日期”不能只放在 skill 文本中。

12.2 模型 Adapter 不止换个 base URL

`ctx.llm.registerAdapter(...)` 让模型能力可替换。Adapter 需要处理模型目录、请求参数、stream chunks、工具调用 JSON 参数片段、usage、finish 和取消等契约。工具参数可能跨多个流片段到达，不能在第一个 delta 就当完整 JSON 解析。19

现有 fixture adapter 是很好的阅读入口：它先发起官方 read 工具调用，再用确定性文本回复，并检查本次 system 中的档案。这种测试不依赖模型随机性，却穿过真实 Session 和工具执行过程。

如果未来接自部署模型，应先验证工具调用、长上下文、取消、模型能力元数据和错误语义，再考虑外观上“能返回一句话”是否达标。对当前项目，默认 provider / model 已收敛由 DSH 管理，报告不建议为了展示可扩展性重新加回一套模型管理 UI。

12.3 Subagent 的适当位置

可以让一个只读子 Agent 汇总近期训练，另一个检查周期安排，再由主 Agent 形成草稿。这种分工可能改善复杂分析，也可能增加 token、延迟和冲突。

不建议多个子 Agent 同时直接修改同一份正式计划。更可控的做法是让它们产出结构化建议，由单一提交者调用领域服务。DSH 的 subagent、workflow 或 job 并不能自动提供 Fitness 文件事务。19

12.4 别把 UI 扩展与模型输出协议混为一层

工具的 canonical value、模型可见文本和 Web UI 卡片是三个层次。业务 ID、状态码、revision 应存在结构化 value 中；给模型解释如何修复的问题放进模型可见内容；浏览器排版由 Client 插件负责。

rc.2 的 Web Client 通过工具名 keyed slot 注册专用展示，不能仅添加 Host `presentCall` / `presentResult` 就期待网页自动出现专用卡片。必须验证历史回放时也能从持久 metadata 重建展示，而不是展示函数运行时重新读某个已经变化的文件。15

13. 怎样增加一个业务工具，又不破坏现有边界

目前项目选择“通用工具 + 工作区导航 + CLI”，这是一个成本低且与现有文件模型一致的方案。是否新增自定义工具，应由重复错误率、调用成本和跨系统复用需求决定，而不是为了把所有 CLI 命令机械翻译成工具。

13.1 一个建议的工具结构

下面是设计示例，不是现有可直接挂载的插件。它假定先定义 `fitnessData` 服务，且服务返回 `{ valid, revision }`；作用是说明 Tool consumer 应该多薄。

```
import { defineTool } from '@deepseek-ai/dsh-tools';

export const inject = ['tools', 'fitnessData'];

export function apply(ctx) {
  ctx.tools.register(defineTool({
    name: 'fitness_validate_plan',
    description: '验证指定日期的正式计划，返回有效性和当前版本。',
    parameters: {
      date: { type: 'string', required: true,
        description: '目标日期，YYYY-MM-DD' }
    },
  },
  {
    output: {
      schema: {
        type: 'object',
        additionalProperties: false,
        properties: {
          valid: { type: 'boolean', required: true },
          revision: { type: 'string', required: true }
        }
      },
    },
    render: (_args, value) => [{
      type: 'text', text: JSON.stringify(value)
    }]),
    async execute(args, exec) {
      return ctx.fitnessData.validatePlan(args.date, exec.signal);
    }
  }));
}
```

真正的 provider 负责日期语义、文件定位、授权、schema 校验、revision 以及读取取消。模型 schema 只能过滤一部分输入，不能证明日期是有效业务日期，更不能授予某个用户访问别人计划的权利。

如果底层仍使用 CLI，provider 应通过受管理的 subprocess 能力和固定 argv 调用它，避免拼接模型生成的 shell 命令。若底层调用 Java，就通过固定 base URL、限时请求和明确错误协议来实现。同一个 consumer 无需了解两种后端差别。

13.2 写工具比读工具多几个合同

建议未来提交工具接收 `expectedProfileRevision`、`expectedPlanRevision` 和 `idempotencyKey`；提交成功返回正式 plan ID / date、版本和领域 outcome。正常冲突应返回可以重读的领域结果，而不是让模型猜测某段异常英文。

工具失败要明确区分：执行前拒绝、输入不合法、版本冲突、远端暂时不可用，以及“提交可能已成功但响应丢失”。最后一种情况最容易导致重复写入，必须靠业务幂等键查询，而不能简单 retry 所有异常。

13.3 什么时候选 CLI，什么时候选 MCP

条件	更合适的选择
单机、文件数据、现有 CLI 稳定	保持当前 CLI 方式
少量高频能力、需要结构化参数和返回	自定义 Cordis Tool + 领域 provider
Java 后端集中管理领域事务	HTTP provider 调用 Java 服务
多个 Agent 产品都需要相同工具	再评估 MCP 作为通用协议
多后端、远程调用、细粒度权限需求明确	MCP 或专门网关，配套授权设计

MCP 不替代事务，也不天然解决 exactly-once。对 training 而言，已有架构约定第一阶段不建设 Fitness MCP 工具矩阵，这个约定与当前规模是匹配的。10

14. 如果接入者是 Java，应该怎么做

14.1 方案一：Java 调 Fitness 业务 API

如果只是触发“生成每日计划”，最直接的是调用现有 Fitness `/api/automation/daily-plan/run-now`，让 Scheduler 保持业务 ownership。注意当前接口等待执行完成再响应，不是一个立即返回任务句柄的异步任务 API。3

这样 Java 不需要知道 DSH Session 细节。但现有接口是本地单用户应用接口；要对外提供服务，还需要明确身份、权限和超时合同。不能把“能发 HTTP”当成“可以直接变成公网业务服务”。

14.2 方案二：Java 调一个稳定的 DSH 业务 bridge

```
Java Application Service / Scheduler
→ 内部 HTTP bridge
→ DSH Session Controller
→ Agent / Tools
→ Java 领域 API 或受控工作区
```

当前 bridge 已经提供最小的 admission / status 形态，所以语言无关。下面是 **Java 17 调用现有 admission 协议的示意片段**，依赖 Jackson；它不包含凭据分发、重试存储和业务提交逻辑，也不代表建议绕过现有 Fitness Scheduler。

```

record Admission(String sessionId, String runId,
                 String requestId, String message) {}

var payload = new Admission(
    "fitness-daily-2026-09-14",
    "example-run-001",
    "fitness-example-run-001",
    "检查当前档案与计划，报告缺少的输入。"
);

var client = java.net.http.HttpClient.newBuilder()
    .connectTimeout(java.time.Duration.ofSeconds(3))
    .build();

var request = java.net.http.HttpRequest.newBuilder(
    java.net.URI.create(
        "http://127.0.0.1:3080/fitness-automation-bridge"))
    .timeout(java.time.Duration.ofSeconds(15))
    .header("Content-Type", "application/json")
    .header("x-fitness-bridge-secret", bridgeSecret)
    .POST(java.net.http.HttpRequest.BodyPublishers.ofString(
        objectMapper.writeValueAsString(payload)))
    .build();

var response = client.send(request,
    java.net.http.HttpResponse.BodyHandlers.ofString());

if (response.statusCode() != 202) {
    throw new IllegalStateException("Admission failed: "
        + response.statusCode());
}

// 这里只完成了 admission。还需解析 accepted、观察执行和核对业务结果。

```

生产化前，建议把 bridge 升级为包含版本、稳定 requestId、Host generation、任务终态和取消的协议，并让 Java 持久化业务任务状态。不要让 Java 模拟浏览器 cookie、直接消费 DSH 私有 UI 内部状态来判定业务完成。

14.3 方案三：Java 拥有业务，DSH 只是 Agent 层

如果目标是企业 Java 服务，可以逐步把训练领域程序迁到 Java：校验与计算、乐观锁、幂等表、审计记录和授权都由 Java 负责；DSH 的工具调用 Java API，浏览器仍可以保留官方 DSH conversation。[建议]

```
DSH Tool: fitness_submit_plan
→ Java controller
→ Application Service
→ 参数 / 领域校验
→ 数据库事务: 检查 revision + 幂等键 + 提交
→ 结构化 outcome
→ DSH 解释结果

Java 业务事件 / outbox
→ Fitness Dashboard 更新
```

这时 DSH Session 仍保存“如何讨论和执行”，Java 数据库保存“最终业务事实”。如果工具调用是远程 HTTP，Java 的本地数据库事务不会跨越整个模型思考过程；应只包住最终确定性提交阶段，不能把数据库连接和锁持有数分钟等待模型。

14.4 方案四：Java 直接管理 DSH stdio 子进程

DSH 有 TypeScript / Python SDK 路径，通常启动 `dsh --profile sdk` 并通过 stdio JSON-RPC 驱动运行。Java 可以另行实现客户端，但需要负责握手、协议 framing、并发请求 ID、通知、stderr 消费、取消、子进程回收和版本匹配。20

这不是当前项目的集成方式。尤其不能在已有 Web Host 旁边又悄悄启动 SDK Host，再假定两者共享活跃 Session 内存。若需要聊天、审批、历史和自动任务统一，本项目当前的同 Host bridge 思路更接近目标。

SDK 路径适合清晰的独立批处理、测试或受监督的专用 Agent worker。是否与 Web 使用同一份持久目录，必须按当前版本的存储及并发契约单独验证。

15. 性能、成本和可观测性

15.1 长期 Host 能节省什么

它避免每次交互都重新装配插件、恢复基础服务和启动浏览器接口；交互与自动任务也能复用同一会话基础设施。但长期存在的 Host 不会让模型推理免费，也不会自动消除上下文读取和工具调用成本。

当前自动任务大约每秒轮询 4 次，一个任务等满 10 分钟，量级接近 2400 次状态查询；实际次数会受到请求耗时影响。单用户本机通常尚可，多任务场景应该评估状态推送、长轮询或带退避的轮询。4

15.2 本地同步 I/O 与远端模型等待要分开测

bridge 的动态 prompt 每次同步读取 settings、profile，并计算哈希；文件小时成本可能很低，文件变大时却会阻塞 Host 的 Node 事件循环。不要只看模型延迟，而忽略了自己在共享进程里做的同步工作。

可以记录这几类指标：Host 启动耗时、admission 耗时、queued 等待、模型请求次数、首 token 延迟、工具耗时、业务校验 / 提交耗时、每任务 token 和重试次数。还应区分界面 ready、Agent idle 和 business outcome，避免一张“成功率”图混合三个不同事件。

15.3 关联标识比完整文本日志更有用

建议建立：

```
occurrence → businessRunId → admissionRequestId  
→ sessionId → turn / tool call → committed plan revision
```

当前两层 runId 分离，bridge 状态又不持久，定位故障时需要额外拼接。下一步先补关联 ID 和结构化错误，比默认记录全部 prompt 更有价值。训练档案和凭据不应因调试而直接进入普通运行日志。

15.4 预算需要覆盖整项任务

模型请求的 maxTokens、工具 timeout、Host 生命周期、Scheduler 重试和任务 deadline 属于不同层。单次请求上限不等于一个任务总上限；子 Agent、反复重试或长工具链也会继续消耗资源。

如果增加预算插件，建议对业务任务记录总模型调用次数、累计 token、累计持续时间和子任务数量；预算触发后要能取消实际执行并给出确定的可恢复状态。不能只让调用方停止等待，而让 Host 在后台继续写文件。

16. 部署与权限：本地成功到上线之间还差什么

16.1 当前 Docker 是同容器双进程应用

镜像内 Fitness 监听 `0.0.0.0:8787`，DSH 通过配置监听 `0.0.0.0:3080`，便于容器端口转发；compose 在宿主上只发布 `127.0.0.1` 的两个端口。`init: true` 帮助进程回收，工作区通过卷挂载，应用根为只读文件系统。¹¹

因此，当前部署既不是每个 Session 一个容器，也不是两个独立可伸缩的微服务。虽然结构上像 sidecar 协作，实际生命周期由一个容器里的 Fitness 父进程管理。

16.2 三种凭据不能互换

凭据	使用方	作用
DeepSeek API Key	LLM adapter / credentials service	访问模型
DSH launch token 与浏览器 cookie	DSH Web Client / Connection	浏览器登录及 Remote 访问
Fitness bridge secret	Fitness service ↔ Host 私有 bridge	后端 admission、状态和密钥管理桥接

当前 bridge secret 默认由 Fitness 生成随机值，通过子进程环境传入 DSH，也可来自显式环境配置。凭据桥通过官方 `credentialsController` 管理 Key，存储路径被定向到工作区 `config/dsh-credentials.yaml`。^{3 6}

DSH 浏览器 cookie 在安装包说明中是 Host-only、HttpOnly、Path=/、SameSite=Strict，绑定 authority；其默认传输场景为本地 HTTP。反向代理上线时应逐项验证 Host / Origin、cookie 属性、token 交换、WebSocket 和 iframe 嵌入，而不是只检查首页返回 200。¹⁸

16.3 为什么不简单挂在 /agent 下

DSH 浏览器接口使用根 `/api` 和 `/api/remote.mux`。如果不完整适配路径、cookie 和资源引用，就把它反代到 Fitness `/agent/`，会与主站 `/api` 发生冲突或导致客户端请求落到错误后端。

已有设计建议独立 Agent origin，例如 `agent.fitness.example.com`；主站为 `fitness.example.com`。这保持两个应用根路径清楚，但仍须验证 cookie、TLS、反代头、WebSocket、刷新和重连。报告不提供一份未经验证就可直接上线的 nginx 配置。²²

16.4 bridge 不自动继承浏览器认证

项目 bridge 直接注册到 `ctx.webServer`，不是通过 `/api` 的浏览器 Remote 注册。admission、status 和 model-settings 各自检查共享 secret；bootstrap route 在自身 handler 中没有这个检查。

bootstrap 返回 workspacePath、workspaceId 和 sessionId，现有测试也以无浏览器 cookie 的直接 fetch 访问它成功。这说明不能写成“所有 DSH 路由都被官方浏览器 cookie 统一保护”。对本地单用户模型，这有其简化背景；对受控网络部署，应明确每条新增 route 的认证和信息暴露边界。6 21

16.5 多用户意味着重新定义 workspace ownership

当前 bridge 的环境变量和 `process.cwd()` 指向一个固定工作区，bootstrap 使用固定交互 Session ID。仅在请求中新增一个 userId 字段，不足以隔离文件、凭据和会话。

多用户方案至少要明确：谁有权创建 / 恢复哪个 Session，Agent 可以读取哪个 workspace，模型凭据按什么范围分配，以及一个用户的任务是否会影响另一个用户的进程资源。每 workspace 独立 Host 可能是可评估的第一步，但它仍需要入口授权和资源调度，不能被描述为已经实现的多租户平台。

17. 已发现的问题与建议优先级

本章是研究发现，不代表本次已经修改实现。证据分成“已复现”“源码确认”和“待专项验证”，避免把潜在风险都写成已经发生的故障。

17.1 优先处理：自动任务完成协议

源码确认：bridge 把同一 Session 的所有 run 记录都随 agent/status 更新。一个历史 run 完成后没有从 Map 中删除；该 Session 后续再次运行时，旧 run 的状态也可能重新变成 running。因此这里的状态更接近“该 Session 的活动状态”，不是某条请求不可逆的独立终态。6

源码确认：AgentRuntime 设置了十分钟外层 deadline，但 admission fetch 和每次 status fetch 本身没有 AbortSignal timeout。一次网络请求长期不返回，就不能保证十分钟到点退出。timeout 抛错后也没有调用 DSH cancel，close() 是空实现，执行可能继续存在。4

建议将 admission 与 execution 分开，给实际请求附加限时信号，增加按请求标识的持久终态与取消，并把 Host generation 纳入协议。对于“无法确认是否完成”的请求，先核对业务提交记录，再决定重试。

17.2 优先处理：重试耗尽后没有持久终止状态

已用隔离探针复现。重试延迟数组是 1 分钟、5 分钟；第三次失败后 retry 字段被清除，而 scheduled_cursor 仍未推进。下一次 tick 再进入同一 occurrence 时，attemptCount 从 1 重新开始。

模拟 UTC 时间	本次结果	保存的 retry attempt	下一次重试时间
01:05:00	failed	1	01:06:00
01:06:01	failed	2	01:11:01
01:11:02	failed	无	无
01:12:03	failed，再次执行	1	01:13:03

探针只使用临时无效档案让 preflight 失败，没有调用模型。结果证明当前不是严格“最多执行三次后永久停住”；不能只看到数组长度为 2，就推断存在可靠的重试上限。5 24

建议对每个 occurrence 持久记录 exhausted / terminal failure，直到人工重试或明确的下一 occurrence 才解除。终止失败与完成业务不应混用同一个成功 cursor。

17.3 优先处理：指令配置作用在了错误层

****配置 dump 与源码共同确认。******Host 的 agent-instructions 行仍 disabled；默认 standard preset 的实际候选包括 AGENTS.md 和 CLAUDE.md。建立 Fitness 拥有的 preset，把候选文件、persona 和工具限制放到真正使用的层。** 17 24

这也说明插件升级后的检查应该覆盖“模型实际看见什么”，不能只验证 YAML 里存在某个配置键。

17.4 第二优先级：可靠性与并发

****源码确认的局限：******Scheduler 在单实例内串行排队，但 claim 中的 lease_expires_at 没有构成多实例互斥检查。既有测试证明能对孤立 claim 做恢复，不等于证明多个 Scheduler 不会同时执行。** 5

****源码确认的局限：******文件审计覆盖 fitness、config、公共 resources 和一个兼容位置的 profile，不是对整个应用和操作系统的完整变更监控；发现越界也不会自动回滚。公共资源不允许写的要求，还应通过权限或 sandbox 强制执行。**

****待专项验证：******bridge 对重复请求在异步 admission 完成前没有 Promise 占位。底层 DSH 有 requestId 去重，但它同样有异步输入处理路径；现有测试覆盖顺序重复，不能推出并发重复严格只执行一次。建议加入并发相同请求、响应丢失、Host 重启等场景。**

****源码确认的局限：******accepted 和 runs Map 没有清理策略。长期 Host 应考虑保留时长、上限或持久存储，不应无限保留所有运行记录。**

17.5 第二优先级：进程管理和配置文档

Host wrapper 通过单个 stdout chunk 正则提取 URL，没有维护跨 chunk 的行缓冲。输出行若被拆分，理论上可能漏掉 ready 信号；应按行解析或采用结构化 readiness 通知。 2

Host 失败后，`/api/dsh-web` 或密钥入口可以触发启动，但当前没有完整的自动后台重启与退避监督协议。启动时 Scheduler 先 start，Host 随后 start，也不能认为第一次定时任务一定等到了 Host 可用。 3

文档 `LOCAL-DEVELOPMENT.md` 仍描述 DSH_PROVIDER / DSH_MODEL 和 settings.model 的旧覆盖方式；当前 wrapper 会过滤两个环境变量，也不生成旧模型选择 patch。compose 仍列出这两个环境变量，容易让维护者误以为设置有效。 2 11 22

Surface 主要使用官方 slot 和 theme API，但源码仍保留对 `[class*="_boot_"]` 的背景处理。因此不能把现状概括成“完全没有依赖上游编译类名”。这属于 UI 升级时应清理或回归的残余耦合。 8

17.6 建议实施次序

1. 明确 admission、run 终态、timeout / cancel 和请求关联 ID。
2. 修复 occurrence 重试耗尽状态，补充并发和重启对账。
3. 建立真正生效的 Fitness preset，收敛指令、工具和动态上下文。

4. 梳理桥接 route 的认证、bootstrap 暴露范围和 Host 健康检查。
5. 再推进独立 Agent origin 的生产部署验证。
6. 有明确业务需求后，再增加 Java provider、少量专用工具或跨应用 MCP。

这个顺序优先保证“知道任务有没有完成、重复了没有、配置是否生效”，再扩大能力面。

18. 本报告的证据与验证边界

本次报告形成过程中已运行以下检查；它们在最终交付前已经完成，不代表后续实现已修改。

实际执行	结果	能说明什么
dsh-installed-host.test.ts	1 项通过，约 17 秒测试执行	真实安装 Host、bridge 认证、顺序去重、每请求档案、官方 read、Chromium 回复和刷新、新会话
automation、runtime、host-model-selection、data-sync、dsh-web-host 五个集成测试文件	12 项通过	当前局部自动化、Host 生命周期与文件同步行为
隔离 config dump	成功	Host 指令行 disabled、默认 standard preset
隔离重试探针	4 次运行记录	重试清空后同 occurrence 再次从 attempt 1 开始

真实 Host 测试使用本地固定回复 LLM adapter，不需要 DeepSeek Key。它证明框架集成链路，不证明真实模型一定遵循教练约束、不证明真实模型业务写入质量，也不证明生产域名、完整审批或所有重连场景已经验收。

未对正式个人档案或会话运行任务；没有为报告修改业务实现。源码发现中未通过故障注入复现的条目已按相应证据强度表述。附录中的建议插件和 Java 片段是说明材料，不声称作为完整生产实现通过了编译或端到端验证。

19. 建议按这个顺序阅读源码

顺序	路径，相对 <code>trainng</code> 根目录	带着什么问题读
1	<code>server/index.ts</code> 、 <code>server/app.ts</code>	谁启动 <code>service</code> 、 <code>Host</code> 、 <code>Scheduler</code> ？
2	<code>server/dsh-web-host.ts</code>	子进程如何创建， <code>profile</code> 怎么生成？
3	<code>dsh-fitness/profile/cordis.patch.yml</code>	项目真正替换了哪些插件？
4	<code>dsh-fitness/automation-bridge/index.js</code>	业务 <code>HTTP</code> 如何进入官方 <code>controller</code> ？
5	<code>server/agent-runtime.ts</code> 、 <code>server/automation.ts</code>	入队、空闲和业务成功怎么分开？
6	<code>server/data-store.ts</code> 、 <code>server/onboarding.ts</code>	校验、 <code>revision</code> 、锁与提交在哪里？
7	<code>server/data-sync.ts</code>	训练数据为什么不随消息结束刷新？
8	<code>src/features/chat/AgentChat.tsx</code>	<code>iframe</code> 如何连接、保留、收起？
9	<code>dsh-fitness/surface/lib/client.js</code>	怎样通过官方 <code>slot</code> 替换壳？
10	安装包 <code>dsh-web-app/cordis.patch.yml</code>	哪些能力从 <code>Host</code> 移到了 <code>preset</code> ？
11	安装包 <code>dsh-agent-presets/presets/standard/agent.cordis.yml</code>	实际 <code>Agent</code> 看见哪些能力？
12	安装包 <code>dsh-api-session-controller/lib/types/commands.js</code>	<code>create</code> 、 <code>prompt</code> 、 <code>requestId</code> 去重如何落地？
13	<code>../deepseek-harness/docs/cordis-primer.md</code> 与 <code>tutorial</code>	依赖、 <code>effect</code> 、 <code>event</code> 、 <code>realm</code> 的框架语义
14	安装包 <code>dsh-tools</code> 、 <code>dsh-system-prompt</code> 的类型声明	如何写不会偏离当前版本的插件？

阅读大型框架时，先沿现有调用链走通，再展开可替换能力，比先按 `packages` 目录逐个读更高效。遇到文档、邻近源码和安装包不一致，要先明确正在讨论哪个版本，再得出结论。

20. 常见问题速查

是不是可以只把 DSH 当一个模型 SDK? 可以选择更简单的模型调用方案，但那不会自动拥有本项目正在复用的 Session、工具、审批、历史和 UI。当前项目接入的明显是完整 Harness。

一个 Session 是否一个进程? 不是。一个长期 Host 承载多个 Session / Agent；工具或外部 provider 可能另外创建进程。

Cordis 是否等于 Spring? 有容器、模块和生命周期方面的相似性，但它还强调可撤销贡献、运行时依赖变化、作用域注册表和事件拦截；不能一一机械翻译。

为什么聊天和自动任务共用 Host? 复用同一会话基础设施和工具能力，避免单独 SDK runtime 带来的双重运行时管理。但这不等于两类任务共享业务结果或自动具备全局写入锁。

为什么看到 idle 还可能失败? idle 只说明 Agent 没有活跃 driver；可能没写文件、写错了、档案过期，或者写到了不允许的位置。

会话历史能当业务数据库吗? 不能。它记录对话和执行；计划、workout 和 metrics 有自己的正式数据与校验。

把业务限制写进 prompt 足够吗? 不足。模型应理解限制，执行层也必须实施权限和提交校验。

加一个插件包就能生效吗? 不一定。还需 profile / preset 挂载，满足依赖，进入正确 scope；浏览器插件还需要正确的 Client 元信息与入口。

改 Host 的同名插件配置会影响 preset 吗? 不能这样假设。本文已经给出了 agent-instructions 的实际反例。

是否应该把 Fitness Scheduler 换成 DSH 内置 schedule? 目前没有这个必要。业务 occurrence、typed outcome 和幂等归 Fitness，增加第二个调度所有者只会让责任变复杂。

Java 最值得复用什么? 复用“Agent 提供语义推理、业务服务决定事实提交”的边界。Java 可以拥有领域事务，DSH 保留 Agent 和会话能力，两者通过明确协议连接。

来源与附录

下列本地路径用于精确定位研究依据。报告本身已包含必要结论与代码节选，离线阅读不依赖源码链接。 `trainng/` 指 `/root/liuchuan/trainng/`； `installed/` 指该仓库 `node_modules/@deepseek-ai/`； `harness/` 指 `/root/liuchuan/deepseek-harness/`。路径后的符号是主要核对位置；各本地资料访问于 2026-09-14。

[1] trainng 版本与依赖。 `trainng/package.json`、`package-lock.json`；Git HEAD；安装包 `dsh`、`cordis`、`tools`、`system-prompt`、`api-session-controller`、`agent`、`base`、`web-app` 的 `package.json`。用于版本、启动脚本与依赖集成结论。

[2] Host 与 profile。 `trainng/server/dsh-web-host.ts`： `start`、`close`、`prepareFitnessProfile`；`trainng/dsh-fitness/profile/cordis.patch.yml`。用于子进程、环境变量、配置生成、插件替换和启动状态。

[3] 应用入口。 `trainng/server/index.ts`、`server/app.ts`： `createLocalService`、`model/settings`、`automation/run-now`、`dsh-web` 路由及启动 / 关闭逻辑。

[4] 自动任务客户端。 `trainng/server/agent-runtime.ts`： `DshHostAgentRuntime.run`、`close`。用于 `requestId`、轮询、`deadline`、返回值和取消局限。

[5] 业务调度。 `trainng/server/automation.ts`： `tick`、`execute`、`resolvePreflight`、`assertAllowedChanges`、`snapshotFiles`。用于 `occurrence`、`claim`、`retry`、领域 `outcome` 与审计。

[6] 私有 bridge。 `trainng/dsh-fitness/automation-bridge/index.js`： `apply`、`bootstrapInteractiveSession`、`readProfileContext`、`parseAdmission`、`matchesSecret`。用于认证、入队、内存状态和上下文。

[7] 主站 Agent UI。 `trainng/src/features/chat/AgentChat.tsx` 及 `src/api/session-history-schemas.ts`。用于 `iframe`、来源校验、握手与界面状态。

[8] 私有 Client Surface。 `trainng/dsh-fitness/surface/package.json`、`index.js`、`lib/client.js`。用于 Client 加载、`layout`、`slots`、`theme`、`history` 和残余 CSS 耦合。

[9] 工作区与变更通知。 `trainng/server/workspace.ts`、`workspace-init.ts`、`data-sync.ts`、`shared/data-sync.ts`。用于目录解析、工作区导航、SSE 和 `fingerprint`。

[10] 领域数据合同。 `trainng/docs/data/FITNESS-DATA-ARCHITECTURE.md`、`server/fitness-cli.ts`、`data-store.ts`、`onboarding.ts`、`agent-settings.ts`。用于正式事实、CLI、`revision`、提交锁和 MCP 边界。

[11] 部署。 `trainng/deploy/Dockerfile`、`compose.yaml`、`README.md`。用于双端口、容器进程、文件复制、凭据和健康检查。

[12] Harness 架构。 `harness/docs/architecture.md`、`packages/boot/app-boot/README.md`；并以 `installed/dsh-base/cordis.patch.yml`、`dsh/lib/profile-boot-Dk-7KqJc.js` 核对当前装配和 CLI 生命周期。

[13] Cordis 插件与生命周期。 `harness/docs/cordis-primer.md`、`docs/cordis-tutorial/02-lifecycle-and-effects.md`、`03-services.md`、`06-composition-and-hmr.md`、`packages/AGENTS.md`。

[14] Cordis 真实实现。 `installed/cordis/src/events.ts`：`isBailed`、`serial`、`waterfall`；`src/context.ts`：`isolate`；`src/service.ts`：服务注册。用于派发和服务域的精确语义。

[15] 工具与 prompt 扩展接口。 `installed/dsh-tools/lib/types/index.d.ts`、`dsh-system-prompt/lib/types/index.d.ts`；`harness/docs/cookbook/adding-a-tool.md`。用于 `guard`、输出协议、动态 `context`、展示和可撤销注册。

[16] Session admission 与 Agent 状态。 `installed/dsh-api-session-controller/lib/types/index.d.ts`、`commands.js`：`create`、`prompt`、`hasPromptRequest`；`installed/dsh-agent/lib/types/runtime-types.d.ts`。用于 `queue`、去重、`idle / running` 与实时事件。

[17] preset 与指令发现。 `installed/dsh-web-app/cordis.patch.yml`、`dsh-agent-presets/presets/standard/agent.cordis.yml`、`dsh-agent-presets/lib/types/index.js`、`mount.js`、`dsh-agent-instructions/lib/index.js`。用于 `standing composition`、`Host / Agent` 层分离和候选文件默认值。

[18] 浏览器传输与认证。 `installed/dsh-client-connection/README.md`：Use this package、Browser authentication and request trust。用于 HTTP、WebSocket、cookie 和 `trusted-host`；文中的公开网络部署仍为建议与待验证事项。

[19] 能力扩展。 `installed/dsh-skill-file-system/README.md`、`dsh-base/cordis.patch.yml`、`standard preset`；`harness/docs/cookbook/adding-an-llm-adapter.md`；`trainng/tests/fixtures/dsh-model/index.js`。用于 `Skill`、模型 `adapter`、`subagent` 和 `workflow`。

[20] SDK 集成对照。 `harness/packages/sdk/client/README.md`、`harness/docs/architecture.md` 的 `Application launch`。用于 `stdio` 子进程模式；未声称本项目安装或使用该 SDK 客户端。

[21] 已执行的集成测试。 `trainng/tests/integration/dsh-installed-host.test.ts`、`automation.test.ts`、`dsh-host-agent-runtime.test.ts`、`host-model-selection.test.ts`、`data-sync.test.ts`、`dsh-web-host.test.ts`。本次共 13 项测试通过，覆盖范围见第 18 章。

[22] 项目设计与历史说明。 `trainng/docs/dsh-integration/DSH-FITNESS-INTEGRATION.md`、`AGENT-AUTOMATION-TECHNICAL-PLAN.md`、`LOCAL-DEVELOPMENT.md`；以源码识别其中现状和历史残留。

[23] DeepSeek 官方发布记录。 DeepSeek Harness v0.1.5-rc.2，官方 GitHub Release，2026-09-10 发布，2026-09-14 访问。用于核实版本发布和预发布身份，不以网页的最新分支替代本地锁定版本。

[24] 本报告隔离探针。 随报告提供 `support/config-probe.mjs`、`config-probe.txt`、`reliability-probe.mts`、`reliability-probe.json`。探针创建并清理临时目录，不访问正式个人工作区。脚本中保留本次机器的应用绝对路径，换机器重跑前应修改该路径；阅读输出不需要执行脚本。